

《高级程序设计》究极模拟卷 C 卷 — 参考答案与解析

题型按最新情报重排（2 分析 + 2 改错）。所有程序输出 / 改错后的“正确版”均经 g++ 13（-std=c++11；分析题按 -fno-elide-constructors）实测验证。红框为高频失分点，蓝框为本卷新变化。

一、简答题

1. this 指针的作用

- 类的成员函数对该类所有对象只有一份代码。每个非静态成员函数都有一个隐含的形参 `this`，类型为“指向当前对象的指针”。
- 通过对象调用成员函数时，编译器把该对象的地址作为实参传给 `this`，成员函数借此知道在操作哪一个对象的数据成员。
- 常见用途：区分同名的成员与形参（`this->x = x;`）、返回当前对象本身（`return *this;`，支持链式调用）。

2. 前置 ++ 与后置 ++ 重载的区别

- **前置 ++a**：声明 `T& operator++();`，先自增再返回自增后的当前对象的引用（可改、可链式）。
- **后置 a++**：声明 `T operator++(int);`（多一个无名 `int` 参数仅用于区分），先保存旧值，自增，再返回旧值的副本（返回值类型是 `T`，不是引用）。

记忆：前置返回引用（新值）、后置多一个 `int` 占位参数、返回值（旧值）。

```
Box& operator++(); Box operator++(int);
```

3. 何时需自定义析构函数；归还哪些资源

- **何时需要**：对象在生命期内申请了需要手动归还的资源（如 `new / new[]` 的内存、打开的文件、网络/锁等），且不会自动释放时，应自定义析构函数在对象消亡时归还。
- **应归还**：用 `new / new[]` 动态分配的内存、对象内部申请的局部资源（文件句柄、动态数组等）。
- **不应（不需要）归还**：程序的全局变量、静态成员变量，以及栈上自动对象——它们不由本对象负责释放。

配套“三法则”：需要自定义析构 / 拷贝构造 / 拷贝赋值之一时，通常三者都要写。

4. STL 三大组成；迭代器的作用

- **三大部分**：容器（container，如 `vector / list / map`）、迭代器（iterator）、算法（algorithm，如 `sort / find`）。
- **迭代器的作用**：是一种“智能指针”，指向容器中的元素，用于对容器元素进行遍历和访问；它把算法与容器解耦——算法通过迭代器操作元素，而不必关心容器的具体类型。

二、程序分析题

T1（继承 + 成员对象 + 对象数组 + `return` 局部 + 值传参）

① 完整输出：

```

===1===
Person(Amy) ctor
Grade(85) ctor
Student(Amy) ctor
Person(Amy) copy
Grade(85) copy
Student copy
Person(Bob) ctor
Grade(90) ctor
Student(Bob) ctor
Person(Bob) copy
Grade(90) copy
Student copy
Student dtor
Grade(90) dtor
Person(Bob) dtor
Student dtor
Grade(85) dtor
Person(Amy) dtor
===2===
Person(Cara) ctor
Grade(88) ctor
Student(Cara) ctor
Person(Cara) copy
Grade(88) copy
Student copy
Student dtor
Grade(88) dtor
Person(Cara) dtor
Person(Cara) copy
Grade(88) copy
Student copy
Student dtor
Grade(88) dtor
Person(Cara) dtor
===3===
Person(Amy) copy
Grade(85) copy
Student copy
-- grading --
Student dtor
Grade(85) dtor
Person(Amy) dtor
===4===
Student dtor
Grade(88) dtor
Person(Cara) dtor
Student dtor
Grade(90) dtor
Person(Bob) dtor
Student dtor
Grade(85) dtor
Person(Amy) dtor

```

② **数组初始化**: `Student arr[2] = { Student("Amy",85), Student("Bob",90) }`; 每个初始化项都是一个临时 `Student`: 先构造临时量 (`Person` ctor→`Grade` ctor→`Student` ctor), 再拷贝构造进数组元素

(`Person` copy→`Grade` copy→`Student` copy), 两个临时量在整条语句结束时按逆序析构。所以这一句共构造 2 个临时 + 2 次拷贝 = 涉及 4 个 `Student` 对象 (2 个最终留在数组里)。

③ `Student c = enroll();` 在 `-fno-elide` 下触发 2 次 `Student` 拷贝构造: `enroll` 里 `return s` (局部 `s` → 返回临时量, 1 次) + 用返回临时量初始化 `c` (1 次)。

===4=== 析构顺序 (按声明逆序): 先 `c` (`Cara`), 再 `arr[1]` (`Bob`), 再 `arr[0]` (`Amy`); 每个 `Student` 内部按构造逆序 **`Student` → `Grade` → `Person`**。

易错点：① 构造永远 **基类 Person → 成员 Grade → 派生 Student**，析构完全倒序；② 数组用临时量初始化 = “临时构造 + 拷贝进数组 + 临时析构”；③ `= enroll()` 是 **2 次**拷贝；④ 多对象按**声明逆序**销毁。

T2（返回值 vs 返回引用 + 链式调用）

① 输出：

```
n=1
n=0
n=1
```

② `incVal()` 返回的是 `*this` 的一份副本（返回类型 `Box`），所以 `a.incVal()` 让 `a.n` 变 1 后，`.dec()` 作用在那个临时副本上（副本 `n` 由 1 减到 0），`a` 本身保持 1 → 输出 `n=1`。

`incRef()` 返回 `*this` 的引用（返回类型 `Box&`），`b.incRef()` 让 `b.n` 变 1 后，`.dec()` 直接作用在 **b 本身**（`b.n` 由 1 减到 0）→ 输出 `n=0`。

③ `c.incRef().incRef().dec()`：两次 `incRef` 都返回对 `c` 的引用，依次把 `c.n` 变成 1、再变成 2；最后 `.dec()` 作用在 `c` 上变 1 → **n=1**。返回引用时链式调用的每一步都操作同一个对象，这正是运算符/方法支持链式调用要返回引用的原因。

易错点：返回值 = 操作副本、原对象不变；返回引用 = 操作本体、可链式。这与 `operator<</code>/operator= 要返回引用是同一个道理。`

三、程序改错题

改错 1（const / static / 成员初始化）

4 处错误与改正：

第 9 行 `id = i;`

错因：`const` 成员不能在构造函数体内赋值，必须在初始化列表中初始化。

改正：`Account(int i) : id(i) { balance = 0; total++; }`

第 15 行 `balance = 0;`

错因：`count()` 是静态成员函数，没有 `this` 指针，不能访问非静态成员 `balance`。

改正：删除该行（静态成员函数只能访问 `static` 成员，如 `total`）。

第 19 行 `balance += 1;`

错因：`getBalance()` 是 `const` 成员函数，承诺不修改对象，不能修改非静态成员 `balance`。

改正：删除该行，直接 `return balance;`

第 24~25 行 `void report(const Account& a) { a.deposit(100); ... }`

错因：`a` 是 `const` 引用，不能调用非 `const` 成员函数 `deposit()`。

改正：去掉形参的 `const`（`void report(Account& a);`）；

或把 `deposit` 设计成不改对象的操作（此处应去 `const`）。

改正后运行输出（已验证）：

```
total=1
balance=100
```

记忆钩子：`const` 成员 → **初始化列表**；`static` 成员函数 **无 this**、只能碰 `static` 成员；`const` 成员函数 **不改** 非静态成员；`const` 对象 / `const` 引用只能调 **const** 成员函数。

改错 2（深拷贝 / `new[]-delete[]` / `operator=` / `operator<<`）

4 处错误与改正：

第 5 / 缺失 类持有裸指针 char* str, 但没有自定义拷贝构造函数。

错因: 默认拷贝构造是浅拷贝, Text b = a; 后 a、b 的 str 指向同一块内存, 析构时同一内存被 delete 两次 (double free)。

改正: 补深拷贝的拷贝构造函数:

```
Text(const Text& t) { str = new char[strlen(t.str)+1]; strcpy(str, t.str); }
```

第 12 行 delete str;

错因: str 是 new char[...] 分配的数组, 应配对使用 delete[]。

改正: delete[] str;

第 15 行 str = t.str; (operator= 内)

错因: 赋值运算符做了浅拷贝, 且未释放旧资源、未做自赋值检测 → 内存泄漏 + double free 隐患。

改正: if (this != &t) { delete[] str;
str = new char[strlen(t.str)+1]; strcpy(str, t.str); }
return *this;

第 19~20 行 operator<< 缺少 return os;

错因: 函数声明返回 ostream&, 却没有 return 语句, 无法链式输出 (行为未定义)。

改正: 在函数末尾加 return os;

改正后运行输出 (已验证):

```
hello hello hello
```

记忆钩子: 类有裸指针就要“三法则”(析构 + 拷贝构造 + 拷贝赋值, 全做深拷贝); new[] 配 delete[]; operator= 四步 (自赋值检测 / 释放旧 / 深拷贝新 / return *this); operator<< 必 return os。

四、编程题

编程 1: Matrix2x2 二阶矩阵

参考实现 (类外定义):

```
// TODO 1 构造: 默认参数只写在声明处; 初始化列表
Matrix2x2::Matrix2x2(double a, double b, double c, double d)
: a(a), b(b), c(c), d(d) {}

// TODO 2 加法: 对应元素相加, 返回新对象; 带 const
Matrix2x2 Matrix2x2::operator+(const Matrix2x2& m) const {
    return Matrix2x2(a + m.a, b + m.b, c + m.c, d + m.d);
}

// TODO 3 乘法: 按矩阵乘法公式
Matrix2x2 Matrix2x2::operator*(const Matrix2x2& m) const {
    return Matrix2x2(a*m.a + b*m.c, a*m.b + b*m.d,
                     c*m.a + d*m.c, c*m.b + d*m.d);
}

// TODO 4 相等: 四元素都相等
bool Matrix2x2::operator==(const Matrix2x2& m) const {
    return a == m.a && b == m.b && c == m.c && d == m.d;
}

// TODO 5 输出: 友元, return os 支持链式
ostream& operator<<(ostream& os, const Matrix2x2& m) {
    os << "[" << m.a << " " << m.b << "; " << m.c << " " << m.d << "];";
    return os;
    // * 必须 return os
}
```

运行输出 (已验证):

```
[1 2; 3 4]
[6 8; 10 12]
[19 22; 43 50]
true
[1 2; 3 4]
```

易错点：① 乘法别写成对应元素相乘——是**矩阵乘法** $a * m.a + b * m.c$ 等；② 默认参数只写在声明处；③ $+/*/=$ 带 `const`、返回新对象；④ `operator<<` 友元 + **return os**。

编程 2：IntList 单向链表（含反转）

参考实现（类外定义）：

```
// TODO 1 析构：先存 next 再 delete
IntList::~~IntList() {
    Node* cur = head;
    while (cur != nullptr) {
        Node* nxt = cur->next;
        delete cur;
        cur = nxt;
    }
}

// TODO 2 头插：新节点指向原头，再更新 head
void IntList::push_front(int v) {
    Node* node = new Node(v);
    node->next = head;
    head = node;
}

// TODO 3 查找：遍历比较
bool IntList::contains(int v) const {
    Node* cur = head;
    while (cur != nullptr) {
        if (cur->val == v) return true;
        cur = cur->next;
    }
    return false;
}

// TODO 4 就地反转：prev / cur / nxt 三指针
void IntList::reverse() {
    Node* prev = nullptr;
    Node* cur = head;
    while (cur != nullptr) {
        Node* nxt = cur->next;    // 先存后继
        cur->next = prev;        // 当前节点反指
        prev = cur;              // prev、cur 前移
        cur = nxt;
    }
    head = prev;                // 新头是原来的尾
}
```

运行输出（已验证）：

```
[ 3 2 1 ]
has 2: true
has 9: false
[ 1 2 3 ]
```

易错点（反转是重点）：① 反转用 **prev / cur / nxt 三指针**，循环里务必先存 `nxt` 再改 `cur->next`，否则丢失后继；② 循环结束 `head = prev`；（新头是原来的尾）；③ 析构先存 `next` 再 `delete`，释放全部节点防泄漏。

本卷相对前两套 / 真题的新变化

① **题型按情报重排**：程序分析由 4 题压到 2 题，新增 **2 道程序改错题**（与你上传的 2017 真题第一大题题型一致），分值 20 分，务必练熟。

② **改错题是全新训练点**：改错 1 打 const/static/成员初始化（2017 真题同款考法）；改错 2 打深拷贝/三法则/delete[]/operator=/operator<<——把你练过的“写对”反过来考“看错”，更难。

③ **分析 T1 加入对象数组**：用临时量初始化数组的“临时构造 + 拷贝进数组 + 临时析构”是你之前没专门练过的，本卷补上。

④ **分析 T2 换新角度**：返回值 vs 返回引用 + 链式调用（2017 incl/inc2 同类），考你对“副本 vs 本体”的理解。

⑤ **编程换题型**：Matrix2x2（考**矩阵乘法**）+ 链表**反转**（三指针就地反转），都与前两套不同。

明早进场前 10 条提醒（针对本卷）

1. 改错必查 5 件套：const 成员→初始化列表；static 函数无 this；const 函数不改成员；const 对象只调 const 函数；new[] 配 delete[]。
2. 类有**裸指针**→三法则全做深拷贝；缺拷贝构造 / operator= 浅拷贝 = double free。
3. operator= 四步：自赋值检测 → 释放旧 → 深拷贝新 → return *this；operator<< 必 return os。
4. 构造顺序 **基类** → **成员对象** → **派生类**；析构完全倒序；多对象按**声明逆序**销毁。
5. 数组用临时量初始化 = 临时构造 + 拷贝进数组 + 临时析构（每个元素一套）。
6. X x = 返回对象的函数()；在 -fno-elide 下是 **2 次**拷贝；值传参 1 次拷贝 + 形参析构。
7. 返回**值** = 操作副本、原对象不变；返回**引用** = 操作本体、可链式（incVal vs incRef）。
8. 前置 ++ 返回引用（新值）；后置 operator++(int) 多一个占位参数、返回值（旧值）。
9. this 指针：成员函数靠它区分“操作哪个对象”；return *this 支持链式。
10. 链表：头插 node->next=head；head=node；；反转三指针先存 nxt；析构先存 next 再 delete，全部释放。